

Currying

Currying = Transforming a function that takes multiple arguments into a sequence of functions that each take one argument.

Instead of:

`f(a, b, c)`

We get:

`f(a)(b)(c)`

Normal Function

```
function add(a, b, c) {  
  return a + b + c;  
}
```

`add(1, 2, 3); // 6`

Curried Version

```
function add(a) {  
  return function (b) {  
    return function (c) {  
      return a + b + c;  
    };  
  };  
}
```

`add(1)(2)(3); // 6`

Why Currying Exists (Real Reason)

Currying is about:

- Partial application
- Function reuse
- Creating specialized functions

- Functional composition
- Cleaner abstraction in large systems

It's not about syntax.

It's about controlling argument flow over time.

When you call:

`add(1)`

You are not computing yet.

You are creating a closure that remembers `a = 1`.

Then:

`add(1)(2)`

Now it remembers:

- `a = 1`
- `b = 2`

Then:

`add(1)(2)(3)`

Now all arguments are available → compute result.

So currying is essentially:

Delayed execution via nested closures.

Why This Is Powerful (Real World)

◆ Example 1: Logging Utility

```
function log(level) {  
  return function(message) {  
    console.log(`[${level}] ${message}`);  
  }  
}
```

```
}
```

```
const errorLog = log("ERROR");  
const infoLog = log("INFO");
```

```
errorLog("Something broke");  
infoLog("Server started");
```

You specialized behavior.

Example 2: API URL Builder

```
function createURL(baseUrl) {  
  return function(endpoint) {  
    return function(id) {  
      return `${baseUrl}/${endpoint}/${id}`;  
    };  
  };  
}
```

```
const api = createURL("https://api.myapp.com");
```

```
const getUser = api("users");  
console.log(getUser(10));
```

Difference Between Currying and Partial Application?

Currying

Always converts:

`f(a, b, c)`

into

`f(a)(b)(c)`

Partial Application

Fixes some arguments.

```
function multiply(a, b, c) {  
  return a * b * c;  
}
```

```
const double = multiply.bind(null, 2);  
double(3, 4);
```

Currying transforms a function into a chain of single-argument functions, while partial application fixes some arguments of a function and returns a new function with fewer parameters.

What Are Limitations of Currying?

Common follow-up.

! Default Parameters Break It

```
function test(a, b = 2, c = 3) {}  
  
test.length = 1
```

So dynamic curry fails.

How Does Currying Relate to Closures?

Correct answer:

Currying relies on closures to remember previously passed arguments across function calls.

Dynamic currying remain